

Next-Generation LPG Distribution System: Architecture & Case Study

High-Detail Analysis of Smart Priority Allocation, Crisis Containment, and Gig Logistics

A Comparative Assessment and Project Justification Report

TECHNICAL RESEARCH & DESIGN INITIATIVE
DEPARTMENT OF DISTRIBUTED LOGISTICS AND AUTOMATION
PUBLISHED MAY 2026

1. Executive Summary

This report provides a comprehensive architectural breakdown and systemic evaluation of our next-generation **LPG (Liquefied Petroleum Gas) Distribution System**. Developed as an alternative to the rigid, flat scheduling models currently deployed by major energy conglomerates, this platform introduces:

- 1. A **Centralized Multi-Tier Priority Allocation Engine** that mathematically guarantees equity of distribution during supply shortages.
- 2. A **Dynamic Crisis Enforcement Model** that automates real-time demand damping and gating during localized fuel shortages.
- 3. An **Automated Pro-Rata Booking Splitter** (Dynamic Backordering) that eliminates inventory locking and partial-order starvation.
- 4. A **Gig-Economy Logistics Portal** that aligns delivery incentives via an open-claiming gig pool, real-time consumer telemetry, and cryptographically verified billing ledgers.

By transitioning LPG distribution from an archaic "First-Come, First-Served" (FCFS) model to a resource-aware, elastic queuing system, our platform prevents resource hoarding, safeguards critical public infrastructure (e.g., healthcare, emergency facilities), and secures delivery tracing from supply warehouse to the consumer's doorstep.

2. Database Schema & Relational Data Models

The system is built upon a highly structured, relational MySQL database schema. To support atomic transactions, audit trails, and strict logistics policies, the schema enforces foreign key integrity, enumerations for compliance states, and generated columns. Below is the exact live schema definition of the core tables:

2.1. Core Node and Consumer Tables

Table: *agencies*

Stores active gas distribution franchises with spatial coordinates for distance-sorted map queries.

Table: *consumers*

Maintains customer profiles, quota levels, and KYC submission/compliance states.

2.2. Transactional and Logistics Tables

Table: `bookings`

The primary transaction ledger. Tracks quantities requested, order statuses, dynamic QR tokens, delivery states, assigned gig agents, and splitting notes.

Table: `supply_stock`

Matches incoming fuel supply lots, enabling granular FIFO matching to prevent stock stagnation.

2.3. Audit and Financial Tables

Table: `allocations`

A high-fidelity audit trail mapping exactly which supply batch was used to satisfy which booking.

Table: `invoices`

Stores legal tax invoices with split calculations for base cost, 9% CGST, 9% SGST, and delivery fees.

2.4. Policy, Crisis, and Logistics Tables

Table: `priority_policies`

Defines standard allocation ranks and emergency limits for customer categories.

Table: `booking_restrictions`

Establishes the standard dynamic gatekeeping rules.

Table: `crisis_events`

Tracks active crisis durations and critical thresholds for local distribution nodes.

Table: `delivery_agents`

Profiles gig-workers and stores their hashed portal credentials.

Table: `shortage_alerts`

Features a generated virtual column (`shortage_amount`) computed automatically as consumer demand exceeds stocks.

2.5. Database Views

To decouple front-end presentation from massive tables joins and optimize read speeds, the system leverages three structured database views:

1. `vw_priority_queue`: Serves as the raw, indexed query source for the Centralized Allocation Engine.
 2. `vw_agency_stock`: Provides real-time tracking of available, allocated, and cumulative stock metrics across all distribution franchises.
 3. `vw_demand_supply_report`: Serves administrative dashboards by grouping pending consumer demands (domestic, essential, commercial) against available supply stock.
-

3. The Problem: Structural Vulnerabilities of Legacy Conglomerate Systems

State-owned and multinational LPG conglomerates (such as *Indane Gas*, *HP Gas*, and *Bharat Gas*) manage networks serving hundreds of millions of consumers. Despite their scale, their distribution algorithms are mathematically primitive, relying on a flat **First-Come, First-Served (FCFS)** queue with static booking constraints. During ordinary supply periods, this model is sufficient. However, during supply-chain disruptions, logistics bottlenecks, or regional crises, FCFS fails catastrophically due to several structural vulnerabilities:

3.1. The Resource Starvation Problem (Lack of Priority Tiers)

In a standard FCFS queue, the system processes bookings strictly chronologically ($t_0 < t_1 < t_2$). During a supply crunch, if high-volume commercial buyers (such as luxury hotels, private clubs, or industrial kitchens) submit bookings at t_0 , the available regional stock is completely depleted.

Consequently, critical domestic households and essential public service providers (such as community clinics, municipal shelters, or public schools) booking at t_1 are starved of supply. Mathematically, the probability of essential services being starved during a shortage approaches 1 as commercial booking volume increases:

$$\lim_{V_{\text{comm}} \rightarrow \infty} P(\text{Domestic Starvation}) = 1$$

3.2. Panic Hoarding Feedback Loops (Static Gating)

During a rumored supply disruption, consumer panic drives a spike in booking velocity ($\frac{dD}{dt}$). Under conventional systems, booking restrictions are static (e.g., maximum 1 cylinder per 15 days, 12 per year).

Because the system cannot scale these constraints dynamically in response to regional stock drops, users place bookings as soon as their minimum gap window opens. This triggers an artificial demand spike that completely dries out buffer stocks, rendering the distributor blind to actual, non-speculative demand.

3.3. The "All-or-Nothing" Inventory Locking Trap

Conventional software architectures handle orders as atomic units. If a consumer requests 2 cylinders, but the local agency has only 1 cylinder remaining in its physical stock:

The Legacy Action: The system either delays the entire booking indefinitely or skips it in the allocation run to avoid a partial-fulfillment exception.

The Consequence: The remaining 1 cylinder sits idle in the warehouse, completely wasted, while the consumer's household has zero cooking fuel. This represents a massive operational inefficiency known as **inventory lock-up**.

3.4. Static Logistics & Gig Worker Disenfranchisement

Existing systems rely on a rigid corps of salarized agency delivery drivers. These drivers operate on fixed schedules and static salaries, meaning they have zero financial incentive to take on high-difficulty routes, perform extra deliveries during crisis states, or optimize their routes.

Additionally, there is no open pool allowing gig workers, independent auto drivers, or freelance logistics workers to claim pending delivery jobs. This leads to delivery backlogs during labor strikes, fuel shortages, or high-demand periods.

4. Core Innovation: The Smart Priority Allocation Engine

Our system replaces the flat FCFS queue with a centralized, priority-ranked, and resource-aware **Allocation Engine** (implemented in `app/allocation/service.py`). The engine executes in a transactional loop, matching available supply batches to queued bookings based on a multi-dimensional priority model.

4.1. Multi-Dimensional Priority Sorting

Rather than processing bookings strictly by time, the engine queries the database view `vw_priority_queue`, which enforces a strict two-stage sorting hierarchy:

1. **Primary Sort key:** `priority_rank` (derived from the database table `priority_policies`).

Domestic (Rank 1): Individual households.

Essential Services (Rank 2): Public hospitals, emergency facilities, defense kitchens, charity shelters.

Commercial (Rank 3): Hotels, restaurants, retail businesses.

2. **Secondary Sort Key:** `booking_date` (strict chronological order `ASC`).

This ensures that during an allocation run, the system will allocate to **every single pending domestic and essential booking** before a single cylinder is distributed to commercial buyers, completely eliminating commercial-driven starvation.

4.2. FIFO Stock Matching & Inventory Tracing

To maintain strict shelf-life tracking and clear financial accounting, the allocation engine matches bookings against specific stock batches rather than a generic inventory counter.

The system fetches available stock batches from the `supply_stock` table ordered by `supply_date ASC`.

Stock is consumed chronologically (FIFO - First In, First Out).

A separate audit trail is written to the `allocations` table, recording precisely which `stock_id` was allocated to which `booking_id`, the exact quantity, and the priority score of the consumer. This audit ledger is vital for regulatory compliance and fraud prevention.

4.3. Dynamic Priority Escalation (Starvation Prevention / Ageing)

In standard multi-tier priority queues, lower-priority tiers (such as Commercial, Rank 3) are vulnerable to **infinite starvation** during extended supply shortages. If new high-priority domestic bookings keep arriving, the engine will continually bypass the commercial bookings, potentially driving local businesses to insolvency.

To mathematically guarantee **liveness** and prevent starvation while maintaining primary equity, our engine implements a **Dynamic Priority Ageing Model**.

1. **The Formula:** For every pending booking, the engine dynamically calculates an **Effective Priority Score** (P_{eff}):

$$P_{\text{eff}} = P_{\text{base}} - \alpha \times \Delta t$$

where:

P_{base} is the standard tier rank (1 = Domestic, 2 = Essential, 3 = Commercial).

Δt is the days elapsed since the booking was created.

$\alpha = 0.02$ is the ageing coefficient (so every 50 waiting days escalates a customer's priority by exactly one full rank tier).

2. **The Sorting Loop:** Before allocating stock, the engine sorts bookings strictly by P_{eff} ASC. If a commercial booking ($P_{\text{base}} = 3$) remains unfulfilled for 50 days, its score escalates to 2.0 (matching a fresh essential booking). After 100 days of waiting, its score escalates to 1.0, moving it ahead of newly placed domestic bookings.

This elegant anti-starvation policy guarantees that **every single valid transaction eventually terminates and gets fulfilled**, showing a level of mathematical operational sophistication that completely surpasses standard conglomerate designs.

5. Deep-Dive: Crisis Management & Dynamic Elastic Rationing

The system features an automated **Crisis Containment Framework** that acts as an emergency governor on the logistics network. When a regional supply chain disruption occurs, an administrator can trigger **Crisis Mode** for a specific agency (`activate_crisis`). This immediately changes the behavior of both the **Booking Gatekeeper** and the **Allocation Engine**.

5.1. The Dynamic Booking Gatekeeper (Booking Phase)

When a consumer attempts to place a booking, the system runs the safety filter `_check_restrictions` (in `app/bookings/service.py`). It detects whether the destination agency is currently flagged as having an active crisis event (`is_active = TRUE` in `crisis_events`). If active, the system dynamically recalculates booking constraints on-the-fly:

1. Gap Days Doubling

Standard safety policies enforce a minimum gap between bookings to prevent rapid re-ordering. Under crisis, this gap is dynamically doubled:

$$\text{\$Min Gap Days (Crisis)} = \text{Standard Gap Days} * 2$$

Example: A domestic user who ordinarily has a 15-day gap restriction must now wait **30 days** before they can book again. This instantly dampens booking velocity and neutralizes panic-buying behaviors.

2. Monthly Quota Halving

The monthly cylinder accumulation limit is dynamically halved to enforce conservation:

$$\text{\$Max Cylinders Month (Crisis)} = \max(1, \text{floor}(\text{Standard Max Cylinders} / 2))$$

Example: A user with a monthly quota of 4 cylinders is restricted to **2 cylinders** during the emergency state.

5.2. On-the-Fly Throttling & Policy Splitting (Allocation Phase)

If a booking is already pending in the queue when a crisis is declared, the allocation engine dynamically throttles the requested quantity during the run:

The `crisis_max` policy values are highly strict:

Domestic: Capped to **2 cylinders**.

Essential Services: Capped to **2 cylinders**.

Commercial: Throttled to **0 cylinders** (a complete freeze on commercial allocation to preserve 100% of remaining fuel for household survival and hospital operations).

Auto-Restrict Split Logic

If a pending booking requested 4 cylinders, but the crisis policy restricts it to 2 cylinders:

1. The engine dynamically updates the database booking record: `cylinders_requested = 2`.

2. It allocates the 2 cylinders immediately so the consumer has essential fuel.
 3. It automatically instantiates a **new pending backorder booking** for the remaining balance of 2 cylinders:
 4. This backorder remains in a **pending** state, locked from allocation until the administrator resolves the crisis mode, at which point the policy constraints revert to standard levels.
-

6. Architectural Breakdown: Dynamic Pro-Rata Booking Splitting

Perhaps the most mathematically advanced feature of our allocation engine is the **Dynamic Pro-Rata Booking Splitter** (also known as the **Shortage Backorder Engine**). It entirely resolves the "All-or-Nothing" inventory locking trap.

6.1. The Algorithm

During the allocation run, if a booking requests a quantity R that is strictly greater than the remaining available regional stock S_{rem} , and $S_{rem} > 0$:

1. The engine calculates the maximum allocatable quantity: $Stake_{now} = S_{rem}$.
2. It calculates the remaining unfulfilled balance: $balance = R - S_{rem}$.
3. It performs a write-transaction that **modifies the existing booking** in-place, updating its requested quantity to $Stake_{now}$.
4. It sets the modified booking's status to **allocated** and ties it to the remaining stock batches.
5. It immediately executes an **INSERT** statement creating a **new, linked pending booking** (the backorder split) for the $balance$ quantity, with a clear audit remark in its **delivery_instructions**.

6.2. Mathematical Trace

Consider an agency with only **2 cylinders** left in available stock.

Two bookings are pending in the queue:

1. Booking #101 (Domestic): Requested **3 cylinders** (Priority Rank 1).
2. Booking #102 (Commercial): Requested **4 cylinders** (Priority Rank 3).

Under our system, the allocation engine executes the following steps:

Step 1: Priority Sorting

Booking #101 is selected first due to its Priority Rank ($1 < 3$).

Step 2: Stock Check vs Demand

The engine detects that Booking #101 requests $R = 3$ cylinders, but remaining stock $S_{rem} = 2$.

Since $R > S_{rem}$, the **Pro-Rata Split** is triggered:

$take_{now} = 2$ cylinders.

$balance = 3 - 2 = 1$ cylinder.

Step 3: Database Database Splits

Original Booking #101 Update:

New Backorder Booking #103 Insertion:

Step 4: Batch Stock Consumption

The 2 cylinders are deducted from the oldest **supply_stock** batch. The remaining agency stock S_{rem} becomes \$0\$.

Step 5: Termination

Since stock is \$0\$, the allocation run terminates. Booking #102 (Commercial) remains untouched in **pending** status.

Result Comparison: Our System vs. Legacy Conglomerates

Legacy Conglomerate: The system would skip Booking #101 (since it cannot satisfy the request of 3). It would also skip Booking #102 (cannot satisfy 4). The 2 cylinders would remain **idle in the warehouse**, and both consumers would have **zero fuel**.

Our Smart Engine: The domestic household receives **2 cylinders immediately** to sustain cooking operations, 0 cylinders sit idle (100% warehouse utilization), and the consumer's remaining 1 cylinder is cleanly queued as a pending backorder to be fulfilled in the very next supply shipment.

7. Gig Logistics Integration & Billing Integrity

To solve the delivery bottleneck, the system features a **Gig-Economy Logistics Portal** (**static/delivery.html** and **app/delivery_agents/service.py**) that acts as a decentralized logistics market.

7.1. The Gig Pool Model

When the allocation engine shifts a booking's status to **allocated**, it is automatically listed in the **Available Jobs Pool**. Independent gig delivery agents (operating similarly to Uber, DoorDash, or Rapido drivers) can view this pool via their portal.

Self-Claiming: Agents claim jobs (**status** moves to **claimed**) based on their route convenience, matching their capacity to unassigned orders.

Customer Telemetry: Claiming opens a real-time support/chat channel between the specific agent and the consumer, allowing direct communication regarding gate codes, drop-off details, or arrival times.

7.2. Commission Ledger Alignment

To incentivize high delivery motivation during periods of high demand or crisis, the platform replaces fixed salaries with a transparent commission structure:

- Every completed delivery dynamically credits a flat **₹50 commission** to the claiming agent's ledger.
- The earnings are tracked directly in the database (`agent_earnings` ledger), derived from the invoice generated at booking. Agents can view their total delivered jobs, cumulative earnings, and historic paystubs in real-time.

7.3. QR Code Cryptographic Billing Integrity

To completely prevent delivery leakage (e.g., drivers claiming they delivered a cylinder while selling it on the black market), the system enforces a strict QR-based physical handshake:

- When a booking is allocated, a secure, unique QR code is generated for that booking.
- Upon physical arrival at the customer's home, the agent **must scan the customer's QR code** using their mobile device.
- The scanning endpoint (`/qr/scan` inside `app/qr/service.py`):
 - Validates the QR token against the database.
 - Confirms the customer's KYC status.
 - Updates the booking status to `delivered`.
 - Unlocks and credits the ₹50 commission** to the agent's ledger in a single atomic database transaction.
 - Marks the associated tax invoice as `paid`.

This closed-loop system guarantees absolute transaction integrity, zero cash leakage, and automated billing compliance.

8. Comparative Analysis: Our System vs. Legacy Conglomerates

The structural advancements detailed above represent a massive leap forward in operational efficiency and social equity.

Functional Dimension	Legacy Conglomerate Systems (Indane, HP, Bharat Gas)	Our Next-Generation Priority & Crisis Platform
Allocation Methodology	Strict FCFS. Starves domestic households when commercial buyers place bulk orders.	Multi-Tier Priority Sorting: Guarantees domestic and essential services are served first.
Shortage Management	All-or-Nothing Block: Orders with partial stock are skipped, leaving inventory idle.	Pro-Rata Splitting: Delivers what is available immediately, queues balance as a backorder.

Functional Dimension	Legacy Conglomerate Systems (Indane, HP, Bharat Gas)	Our Next-Generation Priority & Crisis Platform
Crisis Mitigation	Slow, manual regional bans. Susceptible to panic-buying runs.	Dynamic Scaling: Automatic doubling of gap days, halving of quotas, and freeze on commercial users.
Logistics Structure	Fixed-salary agency drivers. Zero motivation to optimize routes during shortages.	Gig-Economy Pool: Freelance claiming, real-time consumer telemetry, and 50/delivery ledger.
Billing & Verification	Manual paper signatures. High rate of cash leakage and black-market diverts.	QR Cryptographic Handshake: Atomic status update, automatic billing validation, and ledger credit.
Outage & Downtime Fallback	Traditional telephone IVR queues. Complete lock-out during internet outages.	WhatsApp Business API Link: Dynamic state preservation, local storage backup, and instant pre-filled templates.

9. Policy Case Study: The 2026 Karnataka LPG Crisis & Our Architectural Superiority

To evaluate the real-world impact of our system, we examine it against recent crisis mitigation efforts. During the March–April 2026 supply disruption in South India (linked to West Asian geopolitical shipping delays), the **Government of Karnataka** intervened to manage commercial LPG scarcity. Comparing their administrative approach to our software-driven architecture highlights significant operational advancements.

9.1. The Karnataka Government Intervention Model

To resolve the commercial shortages, the Karnataka state administration introduced a manual **4-Tier Standard Operating Procedure (SOP)**:

- 1. **Priority 1 (100% Quota):** Essential public services, including healthcare centers, student hostels, and municipal shelters.
- 2. **Priority 2 (High Quota):** Public utility canteens (at transit hubs), airports, and state offices.
- 3. **Priority 3 & 4 (Rationalized Quota):** Industrial kitchens, agriculture nodes, private hotels, and restaurants.

To enforce this, the government deployed police and regional civil supply departments to conduct physical raids on commercial premises, seizing over 1,800 domestic cylinders diverted to commercial kitchens. Additionally, they urged domestic consumers to utilize the online Delivery Authentication Code (DAC) system.

9.2. How Our Platform Optimizes and Automates This Policy

While the Karnataka government's policy goals were identical to our system's core priorities, their execution relied on **manual, reactive, and resource-intensive administrative enforcement**. Our system implements these objectives programmatically:

Real-time Software-Enforced Gating vs. Manual Directives:

Under the state model, oil companies and local distributors had to manually adjust deliveries based on verbal/written SOP directives—a slow process prone to regional evasion. In our platform, the moment an administrator triggers "Crisis Mode," the entire database-backed booking layer immediately clamps: standard gap days are doubled, commercial caps are set to zero, and domestic limits are halved.

Cryptographic Verification vs. Resource-Heavy Police Raids:

The government had to physically raid restaurants to find domestic cylinders being sold illegally. Our system **prevents diversion at the source**. The delivery agent cannot deliver a domestic cylinder to a commercial business because the agent's application requires a smartphone physical **QR-code handshake** generated only by the domestic customer's verified KYC account. Without the domestic customer's physical QR code, the transaction cannot be closed, and the driver is denied their **■50 commission**, completely neutralizing the financial incentive for black-market diversion.

Pro-Rata Balancing vs. Direct Allocation Starvation:

Under the government's manual SOP, if a Priority 1 facility requested 5 cylinders, but only 2 were available in the local agency, the order would remain pending, leaving the facility empty-handed, or would require a manual override. Our engine dynamically splits the booking, delivering the 2 immediately and placing the remaining 3 in a secure, pending backorder queue.

10. The Engineering Case for a Scratch-Built Platform (Why Patching Fails)

A central question arises in modern logistics design: **"Why build a complete, integrated full-stack application from scratch when we could just patch the allocation logic in existing conglomerate software?"**

While patching seems simpler, it is architecturally unviable due to deep structural flaws in existing monolithic enterprise architectures:

10.1. The Legacy Monolith and Siloed Architectures

Legacy conglomerates (like IOCL or HPCL) manage logistics on mainframe architectures and ancient monolithic ERPs. These systems are highly rigid; their databases are siloed into isolated modules (Logistics, Billing, Customer Portal, and Agency Management are entirely separate databases).

Patching an allocation engine inside one database would not solve the crisis, because the other databases would remain uncoordinated. If you patch the allocation logic, the customer portal would still allow over-booking (triggering customer panic), the billing system would not support dynamic GST-split recalculations on pro-rata splits, and the delivery agents would have no portal to claim gig-delivery jobs.

Our scratch-built application **integrates these four distinct layers into a single, unified database transaction layer**.

10.2. High-Concurrency WebSockets and Real-time Telemetry

Monolithic legacy mainframes are built for batch processing, not high-concurrency real-time interactions. They cannot support:

- A **Decentralized Gig Claiming Pool** where hundreds of freelance auto drivers dynamically poll and claim allocated jobs in real-time.

- Live Customer-Agent WebSockets Chat** allowing the domestic user to guide the delivery agent directly to their door.

Connecting these modern, high-frequency web services to an old mainframe requires massive middleware wrappers, introducing high latency, security vulnerabilities, and single points of failure.

10.3. The Cryptographic Handshake and Invoicing Coordination

In a patched system, completing a delivery is a multi-step, delayed process (often closing the day after delivery). Our system matches the physical scan of a QR code to the immediate release of agent commission and automatic invoice status updates *in a single database transaction session*. This guarantees transaction consistency: a driver cannot claim a commission payout unless the database can mathematically verify the QR-token handshake matching a valid booking and a verified KYC record.

10.4. Outage and Server-Down Resilience (WhatsApp Fallback)

If the central system experiences database downtime or regional connectivity issues, a patched system simply crashes, taking down the entire booking channel. Our full-stack approach co-designs the frontend and backend with client-side state preservation. When the app detects a backend timeout, it immediately activates the **WhatsApp Business API widget**, generating a pre-filled, cryptographically consistent booking string from the browser's local storage cache, preserving the channel with zero data loss.

11. Technical Project Justification & Social Impact

Building and deploying this system is fully justified by three critical dimensions of impact:

11.1. Humanitarian & Social Equity Impact

In emerging economies, cooking gas is not a luxury; it is a fundamental driver of public health and domestic stability. When supply chains collapse, FCFS queues allocate gas to commercial restaurants while households resort to burning biomass (wood, coal), causing severe indoor air pollution. By prioritizing domestic users and essential service providers (hospitals) during crises, the system protects public health and maintains basic social order.

11.2. Operational Efficiency & Asset Utilization

By automating pro-rata splitting, the engine ensures that **no inventory lies idle**. Every single cylinder arriving at a regional warehouse is immediately matched, allocated, and dispatched to the highest-priority buyer in the region. This increases warehouse turnover rates, reduces storage overheads, and maximizes logistics throughput.

11.3. Gig Economy Empowerment

The integration of a gig worker portal creates a transparent, friction-free earning channel for independent drivers and auto operators. By linking physical QR scans directly to digital ledger credits, the system fosters trust, guarantees immediate payout tracking, and provides flexible employment opportunities to local communities.

12. Conclusion

The next-generation LPG Distribution System is a major advancement over legacy energy logistics platforms. By replacing static, flat FCFS models with priority-ranked queues, dynamic crisis scaling, pro-rata split backordering, and secure gig-worker commission structures, it establishes a resilient blueprint for resource distribution. It is a highly secure, socially equitable, and operationally superior platform that fully justifies its development and regional deployment.